

torch.fft.fft#

<https://pytorch.org/docs/stable/generated/torch.fft.fft.html>

Description:

Computes the one dimensional discrete Fourier transform of input. The Fourier domain representation of any real signal satisfies the Hermitian property: $X[i] = \text{conj}(X[-i])$. This function always returns both the positive and negative frequency terms even though, for real inputs, the negative frequencies are redundant. `rfft()` returns the more compact one-sided representation where only the positive frequencies are returned. Supports `torch.half` and `torch.chalf` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension.

Function Signature

```
torch.fft.fft(input, n=None, dim=-1, norm=None, *, out=None)
→ Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> t = torch.arange(4)
>>> t
tensor([0, 1, 2, 3])
>>> torch.fft.fft(t)
tensor([ 6.+0.j, -2.+2.j, -2.+0.j, -2.-2.j])
```

```
>>> t = torch.tensor([0.+1.j, 2.+3.j, 4.+5.j, 6.+7.j])
>>> torch.fft.fft(t)
tensor([12.+16.j, -8.+0.j, -4.-4.j,  0.-8.j])
```

Notes & Warnings

NOTE

Note The Fourier domain representation of any real signal satisfies the Hermitian property: $X[i] = \text{conj}(X[-i])$. This function always returns both the positive and negative frequency terms even though, for real inputs, the negative frequencies are redundant. `rfft()` returns the more compact one-sided representation where only the positive frequencies are returned.

NOTE

Note Supports `torch.half` and `torch.chalf` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension.

Detailed Documentation

Documentation

Computes the one dimensional discrete Fourier transform of input.

The Fourier domain representation of any real signal satisfies the Hermitian property: $X[i] = \text{conj}(X[-i])$. This function always returns both the positive and negative frequency terms even though, for real inputs, the negative frequencies are redundant. `rfft()` returns the more compact one-sided representation where only the positive frequencies are returned.

Supports `torch.half` and `torch.chalf` on CUDA with GPU Architecture SM53 or greater.

However it only supports powers of 2 signal length in every transformed dimension.

input (Tensor) – the input tensor

n (int, optional) – Signal length. If given, the input will either be zero-padded or trimmed to this length before computing the FFT.

dim (int, optional) – The dimension along which to take the one dimensional FFT.

norm (str, optional) –

Normalization mode. For the forward transform (fft()), these correspond to:

"forward" - normalize by $1/n$

"backward" - no normalization

"ortho" - normalize by $1/\sqrt{n}$ (making the FFT orthonormal)

Calling the backward transform (ifft()) with the same normalization mode will apply an overall normalization of $1/n$ between the two transforms. This is required to make ifft() the exact inverse.

Default is "backward" (no normalization).

out (Tensor, optional) – the output tensor.